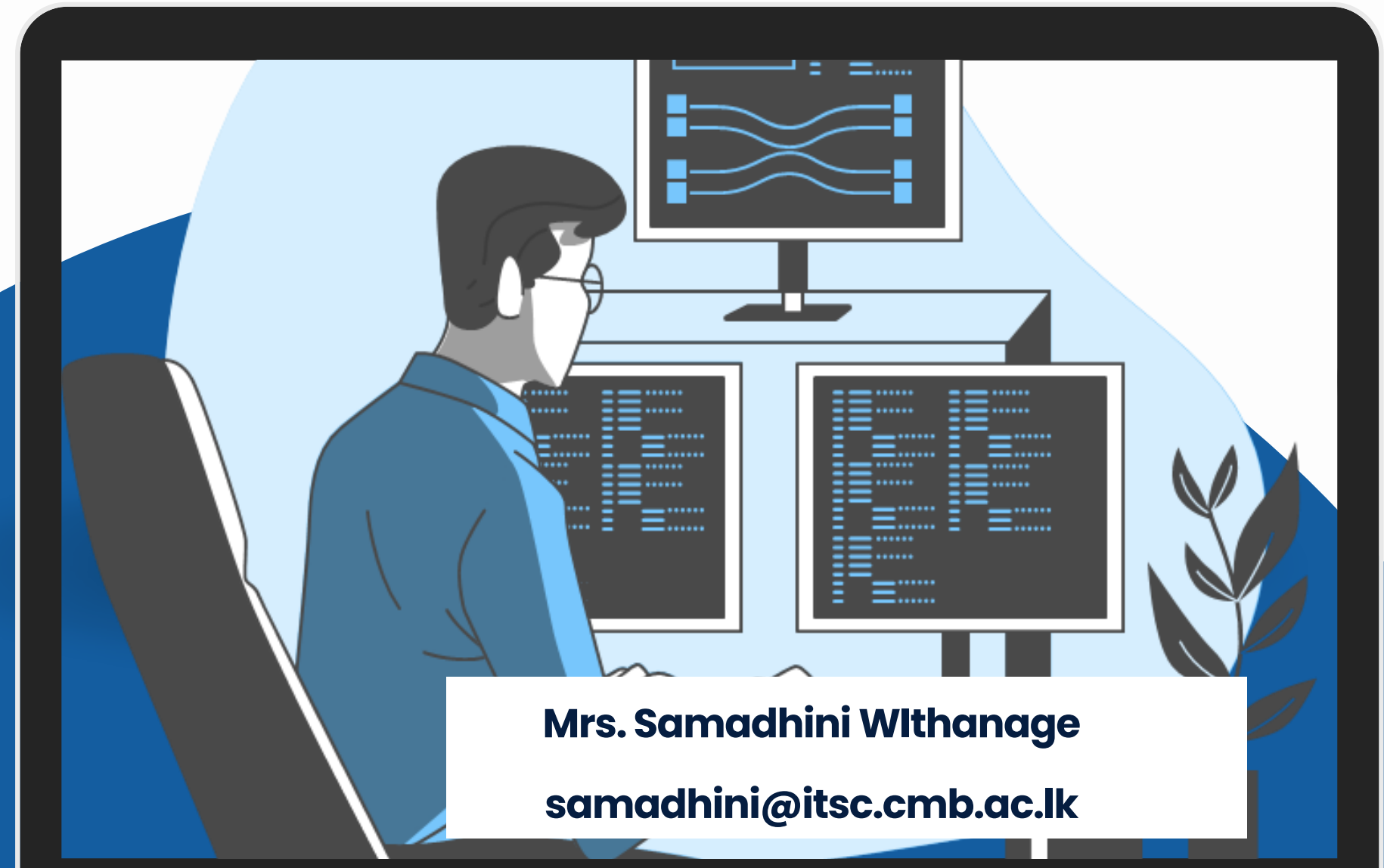


IT 3003

Advanced Programming Techniques



Mrs. Samadhini Wlthanage

samadhini@itsc.cmb.ac.lk



Lesson 4

Advanced Java Class Design & Relationships

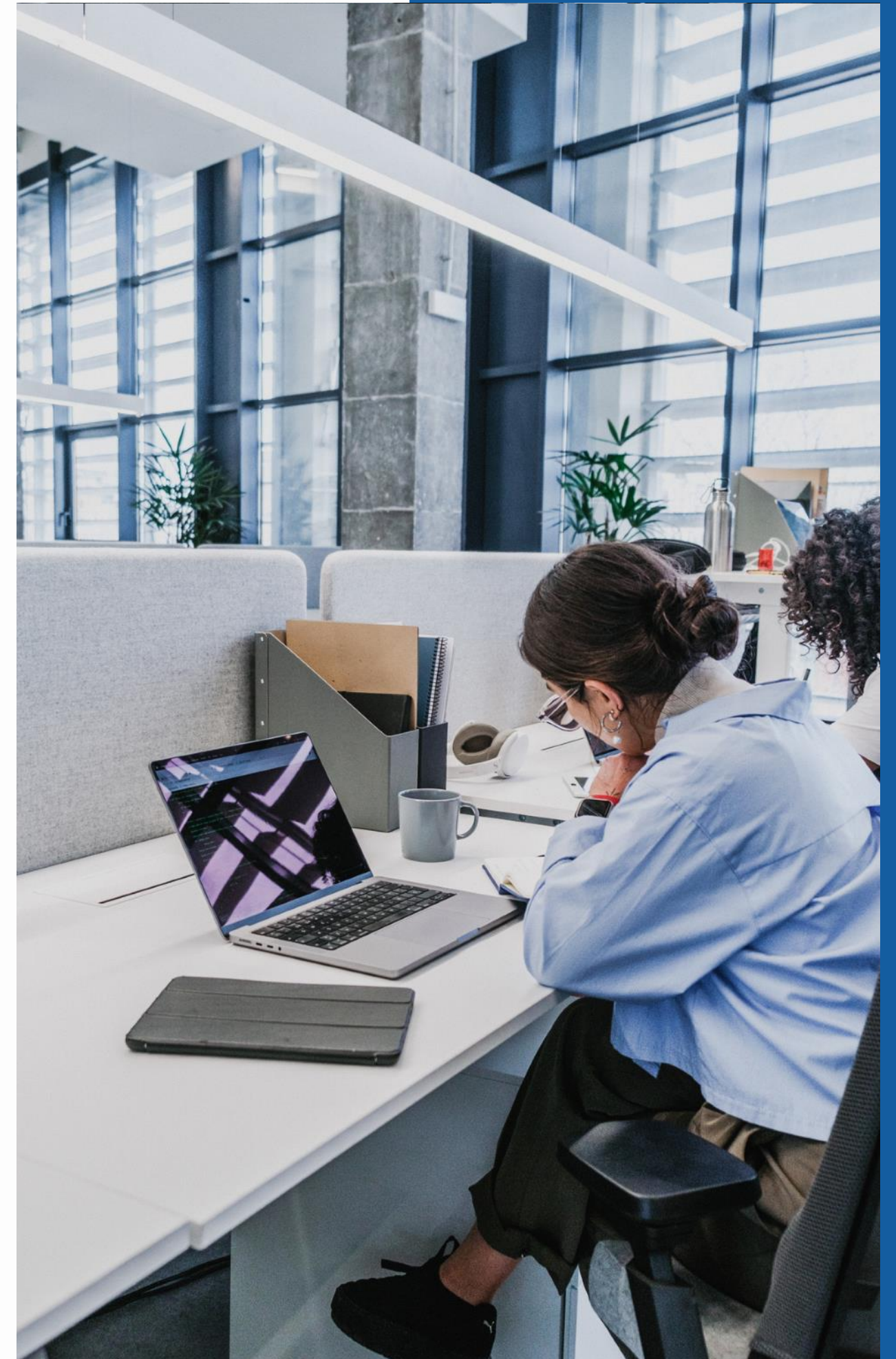
Lesson Outline

▶ Class Restrictions & Safety

▶ Object Relationships

▶ Software Design Quality

▶ Program Organization



Class Restrictions & Safety

In Java, **Class Restrictions & Safety** refers to a set of features that control how classes, objects, and methods can be used or modified. These features are designed to make programs **secure, reliable, and easy to maintain**.

Java is an object-oriented programming language, and many developers may work on the same project. Without proper restrictions, one programmer could accidentally modify important parts of the code, causing errors or security problems. To avoid this, Java provides special mechanisms such as:

- Final Classes & Methods
- Immutable Classes
- Non-access Modifiers

These features act like **protective rules** inside a program.

Class Restrictions & Safety cont..

Why Class Restrictions & Safety are Important

01. Protects Classes and Methods

Some classes or methods **should not be changed** because they contain important business logic or security-related operations.

Java allows developers to:

- Stop inheritance of a class
- Prevent method overriding
- Restrict direct modification of object data

This ensures that important functionality remains safe.

Class Restrictions & Safety cont..

Why Class Restrictions & Safety are Important

02. Controls Object Behavior

Restrictions help define how objects behave in a program.

Without restrictions:

- Objects may change unexpectedly
- Methods may behave differently after overriding
- Program output may become inconsistent

Using safety features ensures predictable behavior.

Class Restrictions & Safety cont..

Why Class Restrictions & Safety are Important

03. Improves Software Security

Security is one of the main reasons for using restrictions.

These features help:

- Protect sensitive data
- Avoid unauthorized changes

Reduce security vulnerabilities

Class Restrictions & Safety cont..

Why Class Restrictions & Safety are Important

04. Improves Reliability and Stability

Reliable software should behave consistently.

Restrictions help by:

- Preventing accidental modifications
- Reducing bugs
- Keeping system behavior stable

This is especially important in:

- Banking systems
- Hospital systems
- Online payment applications
- Airline reservation systems

Class Restrictions & Safety cont..

Why Class Restrictions & Safety are Important

05. Helps Developers Build Maintainable Applications

Large software projects may contain thousands of classes.

Restrictions make code:

- Easier to understand
- Easier to debug
- Easier to maintain

Developers can safely reuse code without worrying about unwanted changes.

Class Restrictions & Safety cont..

Mechanisms Used for Restrictions & Safety in Java

01. Final Classes & Methods

The final keyword is used to restrict modification.

It can be applied to:

- Variables
- Methods
- Classes

Class Restrictions & Safety cont..

▶ Mechanisms Used for Restrictions & Safety in Java cont..

01. Final Classes & Methods cont..

I. Final Class : **A final class cannot be inherited.**

Advantages

- Provides security
- Prevents unwanted inheritance
- Keeps implementation fixed
- Improves reliability

```
final class Test {  
    void display() {  
        System.out.println("Hello");  
    }  
}
```

```
class Child extends Test {    // Error  
}
```

Class Restrictions & Safety cont..

▶ Mechanisms Used for Restrictions & Safety in Java cont..

01. Final Classes & Methods cont..

II. Final Method : **A final method cannot be overridden by subclasses.**

Advantages

- Prevents changing important logic
- Maintains consistent behavior
- Increases application security

```
class Parent {  
    final void show() {  
        System.out.println("Final Method");  
    }  
}
```

```
class Child extends Parent {  
  
    // Error  
    void show() {  
    }  
}
```

Class Restrictions & Safety cont..

▶ Mechanisms Used for Restrictions & Safety in Java cont..

01. Final Classes & Methods cont..

III. Final Variable: Variable whose **value cannot be changed once it is assigned**. It becomes a constant in the program.

Advantages

- Prevents Accidental Value Changes
- Improves Program Security
- Makes Code More Reliable
- Useful for Constants

```
class Test {  
    final int MAX = 100;  
  
    void display() {  
        // MAX = 200;    // Error  
        System.out.println(MAX);  
    }  
}
```


Class Restrictions & Safety cont..

▶ Mechanisms Used for Restrictions & Safety in Java cont..

02. Immutable class

An immutable class is a class whose objects cannot be changed after creation.

Once an object is created:

- Its data remains fixed
- No modifications are allowed

Class Restrictions & Safety cont..

▶ Mechanisms Used for Restrictions & Safety in Java c

02. Immutable class cont..

Characteristics of Immutable Classes

- Class is declared **final** - Prevents inheritance.
- Variables are **private** - Protects direct access.
- No **setter methods** - Prevents modification after object creation.
- Variables are **initialized through constructor** - Values are assigned only once.

```
final class Student {  
  
    private final int id;  
    private final String name;  
  
    Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    int getId() {  
        return id;  
    }  
  
    String getName() {  
        return name;  
    }  
}
```

Class Restrictions & Safety cont..

▶ Mechanisms Used for Restrictions & Safety in Java cont..

02. Immutable class Cont..

Advantages

- More secure
- Thread-safe
- Easier to maintain
- Prevents accidental data changes

Class Restrictions & Safety cont..

▶ Mechanisms Used for Restrictions & Safety in Java cont..

03. Non-access Modifiers

Non-access modifiers are keywords that change the behavior of classes, methods, and variables.

Unlike access modifiers (public, private, etc.), these modifiers do not control visibility.

Instead, they control:

- Inheritance
- Modification
- Memory behavior
- Object creation

Class Restrictions & Safety cont..

▶ Mechanisms Used for Restrictions & Safety in Java cont..

03. Non-access Modifiers cont..

Common Non-access Modifiers in Java

Modifier	Purpose
<code>final</code>	Prevents modification
<code>static</code>	Belongs to class instead of object
<code>abstract</code>	Defines incomplete classes/methods
<code>synchronized</code>	Controls multithreading access
<code>transient</code>	Skips variable during serialization
<code>volatile</code>	Ensures latest value in threads

Object Relationships

What are Object Relationships?

In Object-Oriented Programming (OOP), objects rarely work alone. They interact with other objects to perform tasks and model real-world systems.

An **Object Relationship** describes:

- How one object is connected to another object
- How objects communicate and collaborate
- How responsibilities are shared between classes

These relationships help developers design software that behaves like real-life systems.

Object Relationships cont..

Why Object Relationships are Important

Object relationships are one of the most important concepts in OOP because they help:

- Build realistic system models
- Improve code organization
- Increase reusability
- Reduce complexity
- Create maintainable applications

Object Relationships cont..

Real-World Understanding

In real life:

- A **Teacher** teaches **Students**
- A **Library** contains **Books**
- A **Car** has an **Engine**
- A **University** has **Departments**

Similarly in Java:

- Objects interact with other objects
- One object may contain another object
- One object may depend on another object

This interaction is called an **Object Relationship**.

Object Relationships cont..

▶ Main Types of Object Relationships

▶ Association

▶ Aggregation

▶ Composition

Object Relationships cont..

▶ Main Types of Object Relationships cont..

I. Association

Association is a relationship where:

- Two objects are connected
- Both objects can exist independently
- One object uses or communicates with another

It represents a general relationship between objects.

Object Relationships cont..

▶ Main Types of Object Relationships cont..

I. Association

Real-World Example

- A **Teacher** teaches a **Student**
- A **Doctor** treats a **Patient**
- A **Customer** places an **Order**

Both objects can exist separately.

For example:

A student can exist without a teacher
A teacher can exist without a student

Object Relationships cont..

▶ Main Types of Object Relationships cont..

I. Association

Characteristics of Association

- Loose relationship
- Objects are independent
- Represents communication between objects

Can be:

- One-to-One → One object related to one object.
- One-to-Many → One object related to multiple objects.
- Many-to-Many → Many objects related to many objects.

Object Relationships cont..

▶ Main Types of Object Relationships cont..

II. Aggregation

Aggregation is a special type of Association.

It represents a **weak “has-a” relationship**.

In aggregation:

- One object contains another object
- Contained object can still exist independently

Object Relationships cont..

▶ Main Types of Object Relationships cont..

II. Aggregation

Real-World Example

A Department has Teachers

But:

Teachers can exist without the department

Teachers may move to another department

So the relationship is weak.

Object Relationships cont..

▶ Main Types of Object Relationships cont..

II. Aggregation

Characteristics of Aggregation

- Weak ownership
- Child object can exist independently
- Represents “HAS-A” relationship
- More specific than association

Object Relationships cont..

▶ Main Types of Object Relationships cont..

III. Composition

Composition is a strong form of Aggregation.
It represents a **strong “has-a” relationship**.

In composition:

- One object completely owns another object
- Child object cannot exist independently

Object Relationships cont..

▶ Main Types of Object Relationships cont..

III. Composition

Real-World Example

A House **has** Rooms

Rooms cannot exist without the house

If house is destroyed, rooms also disappear

This is Composition.

Object Relationships cont..

▶ Main Types of Object Relationships cont..

III. Composition

Characteristics of Composition

- Strong ownership
- Child object depends on parent
- Lifetime dependency exists
- Strong “HAS-A” relationship

Object Relationships cont..

► Main Types of Object Relationships cont..

Difference Between Association, Aggregation & Composition

Feature	Association	Aggregation	Composition
Relationship Type	Uses	Has-A	Strong Has-A
Ownership	No ownership	Weak ownership	Strong ownership
Independent Existence	Yes	Yes	No
Object Lifetime Dependency	No	No	Yes
Strength	Weak	Medium	Strong

Object Relationships cont..

▶ Main Types of Object Relationships cont..

Object relationships help:

- Design cleaner systems
- Improve modularity
- Increase reusability
- Reduce duplication
- Represent real-world scenarios effectively

They are heavily used in:

- Banking systems
- School management systems
- Hospital systems
- E-commerce applications

Software Design Quality

What is Software Design Quality?

Software Design Quality refers to how effectively and efficiently a software system is designed.

It measures:

- How well classes and objects are organized
- How easily the software can be maintained
- How flexible and reusable the code is
- How simple or complex the system becomes

A high-quality software design helps developers create programs that are:

- Easy to understand
- Easy to modify
- Easy to test
- Easy to expand in the future

Software Design Quality cont..

Why Software Design Quality is Important

01. Improves Maintainability

Software often needs updates and improvements.

Good design makes it easier to:

- Fix bugs
- Add new features
- Modify existing functionality

02. Reduces Program Complexity

Large applications can contain:

- Thousands of classes
- Many modules
- Multiple developers

Good design organizes the system properly and reduces confusion.

Software Design Quality cont..

Why Software Design Quality is Important

03. Increases Reusability

Well-designed classes can be reused in multiple projects.

Example:

- Login modules
- Payment modules
- Database utilities

This saves:

- Development time
- Cost
- Effort

Software Design Quality cont..

Why Software Design Quality is Important

04. Improves Team Collaboration

In real-world projects:

- Many developers work together
- Different teams handle different modules

Good design helps developers understand code easily.

05. Makes Software More Flexible

Flexible software can adapt to changes without affecting the entire system.

Software Design Quality cont..

Main Concepts

 Cohesion

 Coupling

Software Design Quality cont..

Main Concepts



Cohesion

Measures how closely related class functions are (Cohesion tells us how well the parts of a class work together to perform a single task.)

- **High Cohesion**
 - Class performs one specific task
 - Easier to maintain
- **Low Cohesion**
 - Class performs unrelated tasks
 - Difficult to understand

Software Design Quality cont..

Main Concepts

Cohesion

Benefits of High Cohesion

- Easier to Understand - The class has a clear purpose.
- Easier to Maintain - Changes affect only one area.
- Easier to Reuse - The class can be reused in other projects.
- Easier to Debug - Problems are easier to locate.
- Reduces Complexity - Code becomes more organized.

Software Design Quality cont..

Main Concepts

Coupling

Measures dependency between classes (The level of dependency between two classes or modules.)

It measures:

- How much one class relies on another class
- How strongly classes are connected together

In simple words - Coupling tells us how dependent one class is on another class.

Software Design Quality cont..

Main Concepts

Coupling

Tight Coupling

- Classes highly dependent
- Difficult to modify

Loose Coupling

- Classes less dependent
- Easier to maintain

Benefits of Loose Coupling

- Flexible design
- Better scalability
- Easier testing

Program Organization

What is Program Organization?

- Organizing Java classes properly
- Helps manage large applications
- Improves readability and maintenance

Main Topics

- Packages
- Import Statements

Program Organization cont..

Package

A package in Java is a mechanism used to:

- Organize related classes and interfaces
- Avoid naming conflicts
- Control access
- Improve code maintainability

Why Packages are Important

Packages help developers:

- Organize large projects
- Manage thousands of classes
- Avoid class name conflicts
- Improve security and access control
- Make programs easier to maintain

Program Organization

Types of Packages in Java

- Built-in Packages → Provided by Java (These contain pre-written classes and interfaces)
- User-defined Packages → Created by programmers

Program Organization



Common Built-in Packages

Package	Purpose
<code>java.lang</code>	Basic language classes
<code>java.util</code>	Utility classes
<code>java.io</code>	Input and output
<code>java.sql</code>	Database connectivity
<code>java.net</code>	Networking

Program Organization cont..

Import Statement ??

Take-Home Exercise

Question 1

Explain the concept of User-Defined Packages in Java and discuss their importance in software development.

Your answer should explain:

- What a package is in Java
- The difference between built-in packages and user-defined packages
- How user-defined packages help organize large applications
- The advantages of packages such as code reusability, maintainability, and avoiding naming conflicts

Discuss a real-world scenario (such as a banking system, hospital management system, or e-commerce application) where multiple user-defined packages could be used to organize classes and modules.

Finally explain:

- How to create a user-defined package in Java
- How package naming conventions are used in professional applications
- The relationship between packages and access modifiers
- How packages improve software design quality and modularity

Take-Home Exercise cont..

Question 2

Discuss the purpose and usage of the import statement in Java with respect to packages.

Your answer should explain:

- What the import statement is and why it is needed
- The difference between importing a single class and importing an entire package
- How Java automatically imports the java.lang package
- How the import statement improves readability and simplifies programming

Discuss a real-world scenario (such as a student management system, library system, or online shopping application) where classes from multiple packages need to be imported and used together.

Finally explain:

- The syntax of different types of import statements
- Advantages and disadvantages of using wildcard (*) imports
- Problems that may occur due to class naming conflicts
- Best practices for using import statements in Java applications

Summary

- ▶ What is a Programming Language
- ▶ Computer Programs
- ▶ Types of Programming Languages
- ▶ Compilation vs Interpretation
- ▶ Programming Paradigms
- ▶ Procedural vs Object-Oriented
- ▶ Debugging & Troubleshooting

The background is a blurred office scene with people working at desks, overlaid with a semi-transparent blue filter. A white rectangular box is centered on the image, containing the text "ANY QUESTIONS ?" in a bold, white, sans-serif font. The word "ANY" is on the top line, and "QUESTIONS ?" is on the bottom line.

**ANY
QUESTIONS ?**



THANK YOU!

